Python Fundamentals - Part 2

Yijie Wu

About Me

- Yijie Wu
- Coding/DSA Instructor at IK
- Ex - Research Scientist at Meta/Facebook
- Ex - Senior DS at CVS Health

Agenda

- **Python Fundamentals -2 (Python Week 2)**
  - Slicing of List and String 30 min
  - Tuples 30 min
  - Sets 30 min
  - Dictionaries 30 min
  - Python Modules - Built-In modules 30 min
  - Map, Filter 30 min
  - Scope of Variables 30 min
  - Call by Value and Call by Reference 30 min

- **Practice Problems (from Leetcode)**
  - 28. Find the Index of the First Occurrence in a String
  - 217. Contains Duplicate
  - 349. Intersection of Two Arrays
  - 169. Majority Element
  - 1. 2 Sum

The format for list slicing is **[start:stop:step]**.

- **start** is the index of the list where slicing starts.
- **stop** is the index of the list where slicing ends.
- **step** allows you to select **nth** item within the range **start** to **stop**.

28. Find the Index of the First Occurrence in a String

Medium 1.5K 105

Bloomberg Amazon Microsoft ···

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

```
Input: haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.
```

Example 2:

```
Input: haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.
```

Solution:

```python
class Solution(object):
    def strStr(self, haystack, needle):
        nl, ml = len(needle), len(haystack)
        if nl == 0:
            return nl
        if ml < nl:
            return -1
        for i in range(ml - nl + 1):
            if haystack[i:i+nl] == needle:
                return i
        return -1
```

Python Tuples - Overview

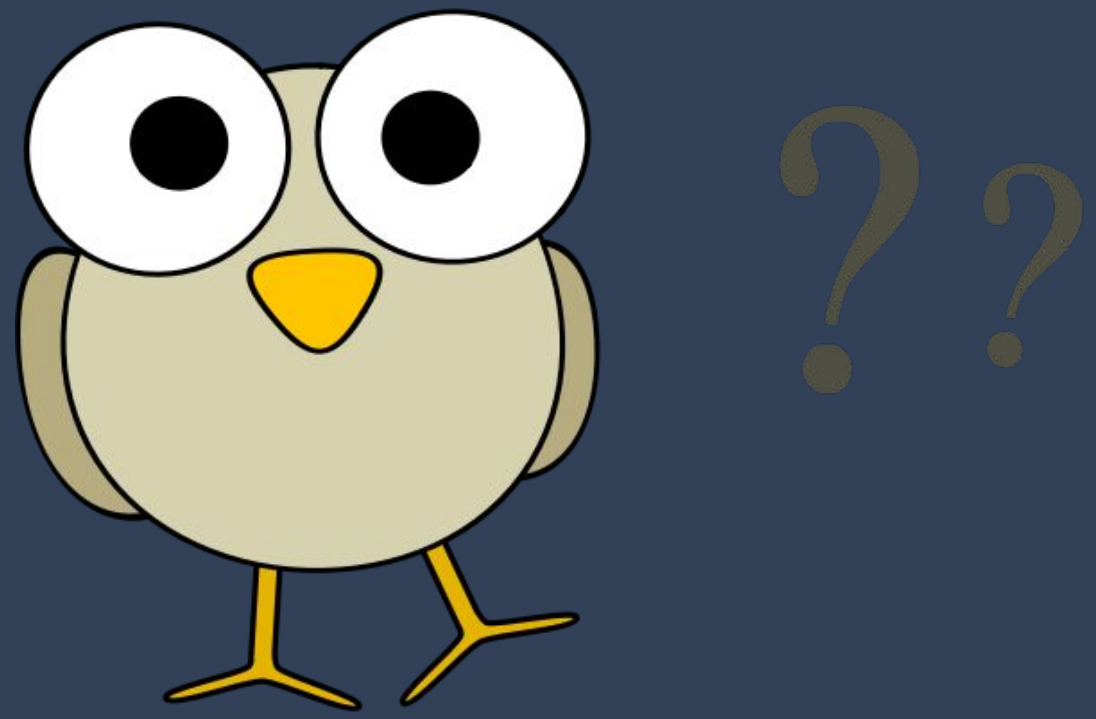
- Tuples:
  - Ordered, immutable collection of elements
  - Similar to lists, but with parentheses instead of square brackets

Python Tuples - Basic Operations

- Basic operations:
  - Accessing elements: tuple[index]
  - Slicing: tuple[start:end]
  - Tuple packing: multiple values assigned to a tuple
  - Tuple unpacking: values in a tuple assigned to multiple variables

```python
# Creating a tuple
my_tuple = (1, 2, 3, "Hello", 3.14)

# Accessing elements of a tuple
print(my_tuple[0])      # Output: 1
print(my_tuple[3])      # Output: Hello

# Slicing a tuple
print(my_tuple[1:4])    # Output: (2, 3, 'Hello')

# Length of a tuple
print(len(my_tuple))    # Output: 5

# Looping through a tuple
for item in my_tuple:
    print(item)

# Nested tuple
nested_tuple = ((1, 2), (3, 4), (5, 6))

# Unpacking a tuple
x, y = (10, 20)
print(x)                # Output: 10
print(y)                # Output: 20
```

Wonder why we use tuples? Why not lists in some cases?



Python Tuples - Usage and Advantages

- Tuples vs. lists:
  - Tuples are immutable, while lists are mutable
  - Tuples use less memory than lists
  - Tuples can be used as keys in dictionaries, while lists cannot
- Common use cases:
  - Returning multiple values from a function
  - Storing related data as a single unit

Python Tuples

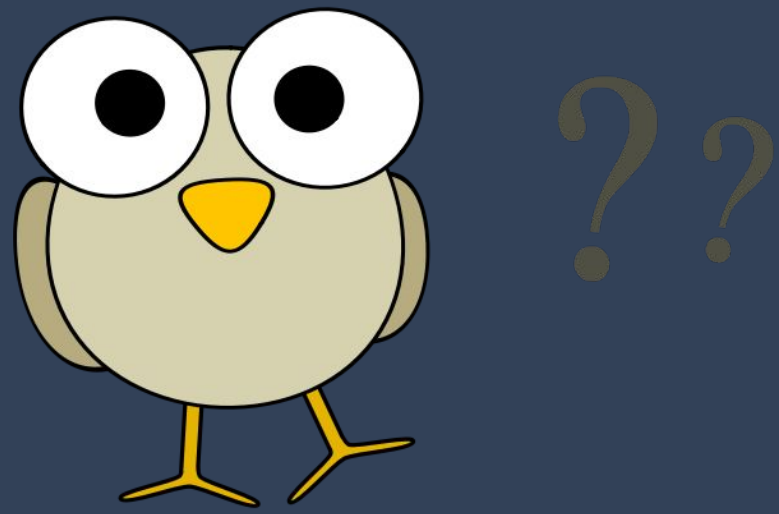
```python
# Using a tuple to store related data
person = ("Alice", 25, "New York"')
name, age, city = person
print (name) # Prints "Alice"
```

```python
# Using a tuple to return multiple values
def divide(a, b):
    quotient = a // b
    remainder = a % b
    return (quotient, remainder)

result = divide (10, 3)
print (result)
# Prints (3, 1)
```

Python Sets - Usage and Advantages

- Sets:
  - Unordered, mutable collection of unique elements
  - Created using braces {} or the set() constructor
  - Cannot contain duplicate elements

Wonder why we use sets when we have so many other data types to use in python?



Python Sets - Usage and Advantages

- Basic operations:
  - Accessing elements: not supported (no indexing)
  - Adding elements: set.add(item)
  - Removing elements: set.remove(item)
  - Set operations: union, intersection, difference

```python
# Creating a set
fruits = {"apple", "banana", "cherry"}

# Adding an element
fruits.add("orange")

# Removing an element
fruits. remove( "banana")

# Set operations
more_fruits = {"orange", "strawberry", "kiwi"}

all_fruits = fruits.union(more_fruits)

common_fruits = fruits. intersection(more_fruits)

print(all_fruits)
print (common_fruits)
# Prints {"apple", "cherry", "kiwi", "orange", "strawberry")
# Prints {"orange"}
```

Sets in Python - Usage and Advantages

- Sets vs. other collections:
  - Sets are faster for membership testing (e.g., checking if an element is in a set)
  - Sets eliminate duplicates automatically

Sets in Python - Usage and Advantages

- Common use cases:
  - Finding unique elements in a list
  - Removing duplicates from a list
  - Performing set operations (union, intersection, difference)

```python
# Using a set to find unique elements
numbers = [1, 2, 3, 4, 3, 2, 1, 5, 6]
unique_numbers = set (numbers)
print (unique_numbers) # Prints {1, 2,
3, 4, 5, 6}


# Using a set to remove duplicates
numbers = [1, 2, 3, 4, 3, 2, 1, 5, 6]
unique_numbers = list(set (numbers))
print (unique numbers)
# Prints [1, 2, 3, 4, 5, 6]
```

217. Contains Duplicate

Easy 8.5K 1.1K

Amazon Apple Bloomberg ···

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

```
Input: nums = [1,2,3,1]
Output: true
```

Example 2:

```
Input: nums = [1,2,3,4]
Output: false
```

Example 3:

```
Input: nums = [1,1,1,3,3,4,3,2,4,2]
Output: true
```

Solution :

```python
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        # Create an empty hash set
        seen = set()
        # Iterate over the array
        for num in nums:
            # If an element is already in the hash set, return True
            if num in seen:
                return True
            # Otherwise, add the element to the hash set
            seen.add(num)
        # If we finish iterating over the array and have not found a duplicate, return
False
        return False
```

349. Intersection of Two Arrays

Easy 4.3K 2.1K

Amazon Google Facebook ···

Given two integer arrays `nums1` and `nums2`, return *an array of their intersection*. Each element in the result must be **unique** and you may return the result in **any order**.

Example 1:

```
Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2]
```

Example 2:

```
Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
Output: [9,4]
Explanation: [4,9] is also accepted.
```

Solution :

```python
def intersection(self, nums1, nums2):
        """
        :type nums1: List[int]
        :type nums2: List[int]
        :rtype: List[int]
        """
        s1 = set(nums1)
        s2 = set(nums2)

        intersection = [x for x in s1 if x in s2]
        intersection = list(set(intersection))
        return intersection
```

Python Dictionary - Overview and Basic Operations

- Dictionaries:
  - Ordered*, mutable collection of key-value pairs
  - Created using braces {} or the dict() constructor
  - Keys must be unique and immutable, values can be of any type

* As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.

Python Dictionary - Overview and Basic Operations

- Basic operations:
  - Accessing values: dictionary[key]
  - Adding or modifying values: dictionary[key] = value
  - Removing values: dictionary.pop(key)
  - Dictionary methods: keys(), values(), items()

If we want to store information about countries and their capitals, we can create a dictionary with country names as **keys** and capitals as **values**.

Keys	Values
Nepal	Kathmandu
Italy	Rome
England	London

```python
# Creating a dictionary
person = {"name": "Alice", "age": 25, "city": "New York"}

# Accessing values
print (person ["name" ])
# Prints "Alice"

# Adding or modifying values
person ["age"] = 26
person ["gender"] = "female"
# Removing values
person.pop("city")
# Dictionary methods
keys = person.keys ( )
values = person.values ( )
items = person.items ()

print (keys)
# Prints dict_keys(["name", "age", "gender"])
print (values)
# Prints dict_values(["Alice", 26, "female"])
print (items)
# Prints dict_items([("name", "Alice"), ("age", 26),
("gender","female" )
```

Wonder why we use Dictionaries, than other collections?

??

Python Dictionary - Usage and Advantages

- Dictionaries vs. other collections:
  - Dictionaries provide fast access to values based on keys (similar to sets)
  - Dictionaries allow for easy and efficient mapping of keys to values

Python Dictionary - Usage and Advantages

- Common use cases:
  - Storing and retrieving configuration settings
  - Counting occurrences of elements in a list
  - Mapping values to keys for efficient lookup

```python
# Using a dictionary to count occurrences of elements
numbers = [1, 2, 3, 2, 1, 1, 4, 5, 6, 5, 5]
counts = {}

for number in numbers:
    if number in counts:
        counts [number] += 1
    else:
        counts [number] = 1


print (counts) # Prints {1: 3, 2: 2, 3: 1, 4: 1, 5: 3, 6: 1}
```

1. Two Sum

Easy 43.1K 1.4K

Amazon Adobe Apple ···

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Solution :

```
#The code uses nested loops to iterate over each pair of numbers in the nums list. The
outer loop runs from 0 to len(nums)-1, and the inner loop runs from i+1 to len(nums)-1,
where i is the current index of the outer loop. Within the nested loops, constant time
operations are performed, such as checking if the sum of two numbers equals the target,
appending indices to the a list, and using break and continue statements.
#Therefore, the overall time complexity of the code is O(n^2) due to the nested loops.

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        a=[]
        for i in range(len(nums)):
            for j in range(i+1,len(nums)):
                if (nums[i]+nums[j]==target):
                    a.append(i)
                    a.append(j)
                    break
        return a
```

169. Majority Element

Easy 14129 435 Add to List Share

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-10^9 <= nums[i] <= 10^9`

Approach:

To solve the problem in linear time and O(1) space, we can use Boyer-Moore Voting Algorithm. The algorithm works by maintaining a count variable and a candidate variable. We iterate through the array, for each element, we check if the count is 0, if it is, we update the candidate variable to the current element. We then increment the count variable if the current element is equal to the candidate, otherwise we decrement the count variable. At the end of the iteration, the candidate variable will hold the majority element.

Complexity
Time complexity:
The time complexity of this algorithm is O(n), where n is the length of the array.

Space complexity:
The space complexity is O(1), as we only need to maintain two variables.

Solution :

```python
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        # initialize the count and candidate variables
        count = 0
        candidate = None

        # iterate through the array
        for num in nums:
            # if count is 0, update the candidate variable
            if count == 0:
                candidate = num
            # increment or decrement count based on whether the current element is the
candidate
            count += (1 if num == candidate else -1)

        # return the candidate variable
        return candidate
```

Python Modules - Overview and Built-in Modules

- Modules:
  - A Python file containing functions, classes, and variables
  - Can be imported and used in other Python files
  - Built-in modules: math, random, datetime, os, etc.

Python Modules - Overview and Built-in Modules

- Using built-in modules:
  - Importing a module: import module_name
  - Using functions from a module: module_name.function_name()

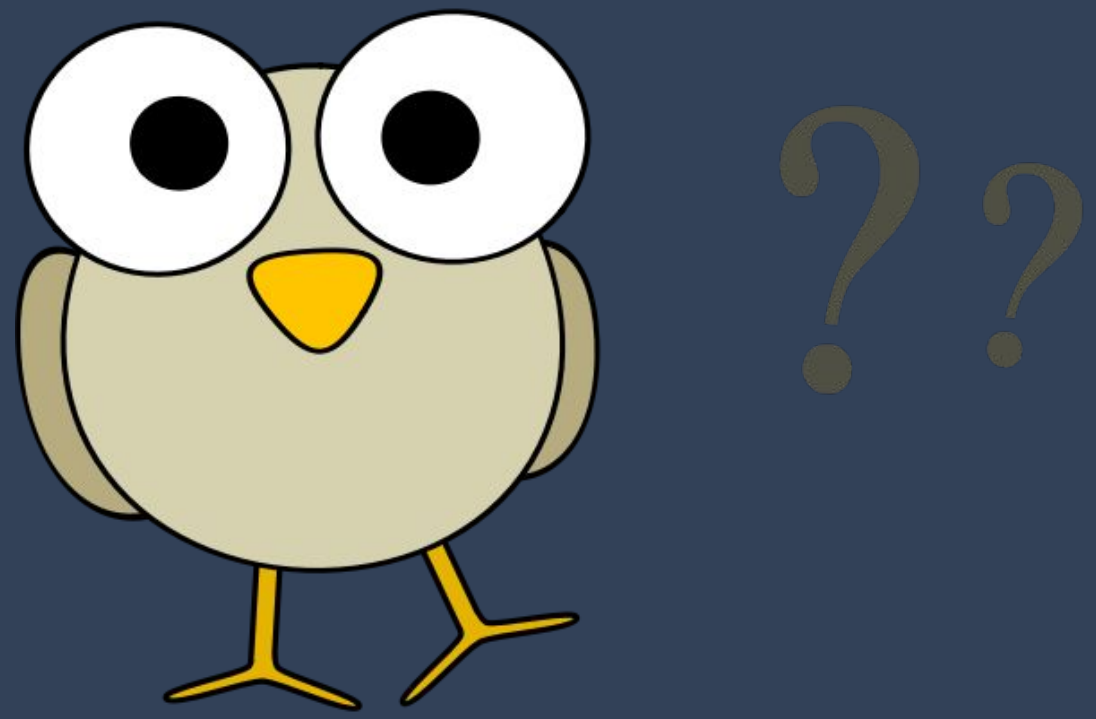
```python
# Using the math module
import math
x = math.sqrt (16)
print(x) # Prints 4.0



# Using the random module
import random
x = random. randint (1, 10)
print(x) # Prints a random integer between 1 and 10 (inclusive)
```

1.

Python Modules - User-defined Modules and Creating a Module

- User-defined modules:
  - Python files created by the user that can be imported and used in other Python files
  - Useful for grouping related code into a single module

Do you know how to create a module in Python?



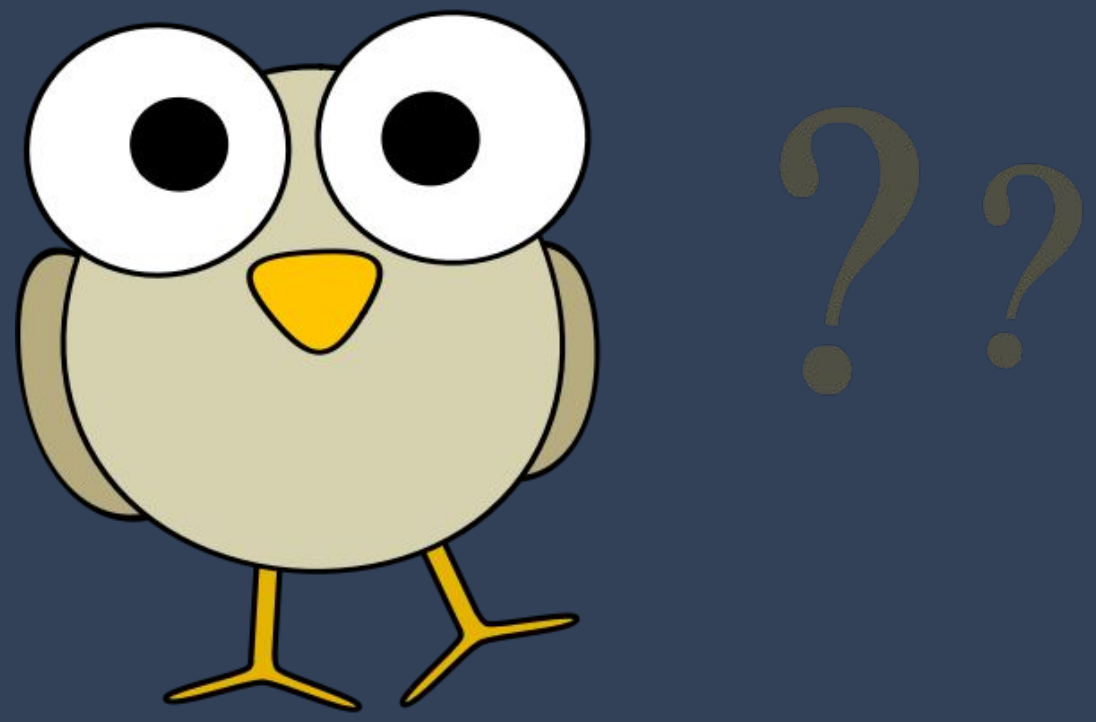
Python Modules - User-defined Modules and Creating a Module

- Creating a module:
  - Write the code in a Python file with a .py extension
  - Use functions, classes, and variables in the file
  - Import the module in another Python file using import module_name

```
# Creating a module called my_module
# Filename: my_module. py
def greet (name) :
    print Hello, ." + name + "|")
```

```
# Using the my_module module in another Python file
import my_module
my_module-greet("Alice") # Prints "Hello, Alice!"
```

Do you know about Python Lambda functions?



Python lambda Function Declaration

We use the `lambda` keyword instead of `def` to create a lambda function. Here's the syntax to declare the lambda function:

```
lambda argument(s) : expression
```

Here,

- `argument(s)` - any value passed to the lambda function
- `expression` - expression is executed and returned

Let's see an example,

```
greet = lambda : print('Hello World')
```

map() Syntax

Its syntax is:

```
map(function, iterable, ...)
```

---

map() Arguments

The `map()` function takes two arguments:

- **function** - a function
- **iterable** - an iterable like sets, lists, tuples, etc

You can pass more than one `iterable` to the `map()` function.

filter() Syntax

The syntax of `filter()` is:

```
filter(function, iterable)
```

---

filter() Arguments

The `filter()` function takes two arguments:

- **function** - a function
- **iterable** - an iterable like sets, lists, tuples etc.

Scope of Variables in Python - Overview and Local Variables

- Scope of variables:
  - The area of the program where a variable is accessible
  - Python has two main scopes: global and local

Scope of Variables in Python - Overview and Local Variables

- Local variables:
  - Defined inside a function or block
  - Only accessible within that function or block
  - Created when the function is called and destroyed when the function ends

```python
# Example of a local variable
def my_function():
    x = 5
    print (x)
my_function() # Prints 5
```

Scope of Variables in Python - Global Variables and Call by Value

Global variables:

- Defined outside of any function or block
- Accessible throughout the entire program
- Can be modified by functions

Scope of Variables in Python - Global Variables and Call by Value

- Call by value vs. call by reference:
  - Call by value: a copy of the value is passed to a function (used for immutable objects)
  - Call by reference: a reference to the original object is passed to a function (used for mutable objects)

Call by value Example

```
# Example of a global variable and call by value
x = 10
def my_function (y):
    y += 5
    print ("Inside function:", y)



my_function (x) # Prints "Inside function: 15"
print("Outside function:", x # Prints "Outside function: 10"
```

Call by Reference Example

```
# Example of call by reference
my_list = [1, 2, 3]
def modify_list(lst):
    1st.append (4)

modify_list (my_list)
print(my _list) # Prints [1, 2, 3, 4]
```